SE1020 | OBJECT ORIENTED PROGRAMMING

Hotel Room Reservation System

Project Development Plan

Group Y1.S2.WD.IT.01.02

Java 17 | Spring Boot 3 | JSP | MySQL | GitHub

TEAM MEMBERS

Student ID	Name	Component
IT26100491	Paramie W. D.	Admin Management
IT26100159	De Silva M.R	Room Management
IT25100116	Dayarathne D. H. P.	Reservation Management
IT26100036	Prabath G.O.G.D.	Booking Management
IT26100027	Dissanayake D.M.S.S	Check-in/Check-out and Guest Management
IT26100009	Wickramarathna B.A.N.U.H	Finance Management

October 2026

Contents

01 Project Overview
02 Team and Responsibilities
03 System Flow
04 Database Design
05 System Architecture
06 Component Plans
07 OOP Concepts
08 Development Workflow
09 Timeline
10 Deliverables

01 Project Overview

The Hotel Room Reservation System is a reception-desk web application that handles a guest's full stay: reservation, booking, check-in, check-out and the final invoice. It runs on localhost for internal reception use. Six members each own one component with full CRUD operations, at least three JSP pages and their own OOP classes.

Key decisions

Area	Decision
Backend	Java 17, Spring Boot 3 (Maven), Spring MVC `@Controller` classes
Views	JSP + JSTL under `WEB-INF/jsp/<module>/`, styled with Bootstrap 5
Data	MySQL on localhost:3306, accessed through hand-written JDBC DAO classes
Login	Session-based login (`HttpSession`) checked by an interceptor
Encapsulation	All getters and setters written by hand (no Lombok)
Room availability	Calculated from reservations and bookings for the requested dates, never stored
Reservation expiry	No automatic expiry; staff convert or cancel after the date passes
Version control	One GitHub repository, one feature branch per member

02 Team and Responsibilities

Each member owns one component, including its CRUD operations, pages and classes. Guest Management is grouped with Check-in and Check-out, since guest details are captured at the front desk. All packages sit under `com.hotel.reservation`.

#	Student ID	Name	Component	Package
1	IT26100491	Paramie W. D.	Admin Management	`admin`
2	IT26100159	De Silva M.R	Room Management	`room`
3	IT25100116	Dayarathne D. H. P.	Reservation Management	`reservation`
4	IT26100036	Prabath G.O.G.D.	Booking Management and project setup	`booking`, `common`
5	IT26100027	Dissanayake D.M.S.S	Check-in/Check-out and Guest Management	`checkinout`, `guest`
6	IT26100009	Wickramarathna B.A.N.U.H	Finance Management	`finance`

Prabath G.O.G.D. also completes the one-time setup: GitHub repository, Spring Boot skeleton, `common` package, database script and the shared JSP layout.

03 System Flow

A stay starts either as a Reservation, where the guest pays an advance, or as a direct walk-in Booking. From Booking onwards both paths are the same, and both end with the final invoice in Finance.

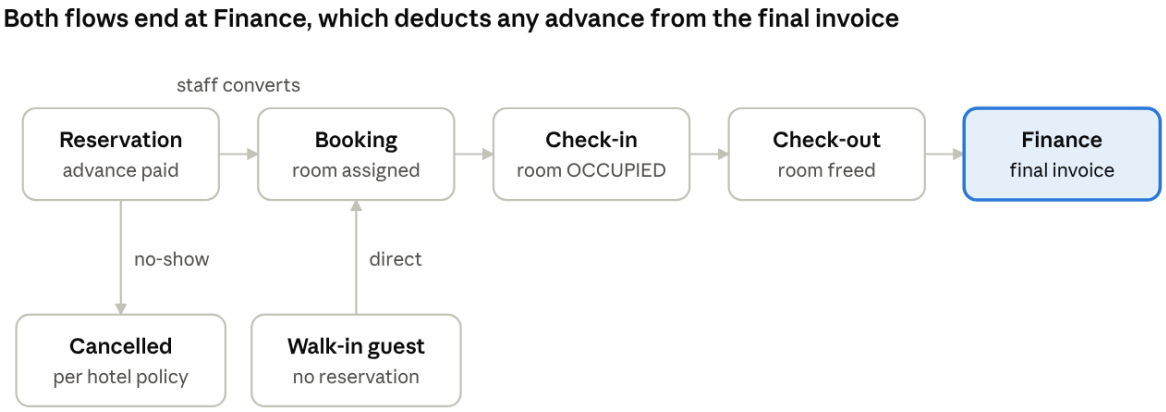


Figure 1. Guest stay flow for reservation and walk-in guests

A Reservation becomes a Booking only when staff convert it on the guest's arrival. A no-show is cancelled by staff, never automatically.

Status lifecycle

Entity	Statuses	Changed by
Room	AVAILABLE, OCCUPIED, MAINTENANCE	Room (maintenance), Check-in (occupied), Check-out (available)
Reservation	PENDING, CONVERTED, CANCELLED	Reservation; Booking sets CONVERTED
Booking	CONFIRMED, CHECKED_IN, CHECKED_OUT, CANCELLED	Booking; Check-in/out sets CHECKED_IN and CHECKED_OUT
Invoice	UNPAID, PAID, VOID	Finance

Room availability rule

A room is available for a requested date range when all three conditions hold:

1. The room status is not MAINTENANCE.
2. No PENDING reservation for that room overlaps the dates.
3. No CONFIRMED or CHECKED_IN booking for that room overlaps the dates.

> **Overlap check**
> Two date ranges overlap when `newFrom < existingTo && newTo > existingFrom`.
> CONVERTED reservations are skipped, so a converted stay is never counted twice.
> Available rooms today = total rooms - rooms that fail the rule for today.

04 Database Design

The system uses one MySQL database, `hotel_db`, with eight tables. The create script is kept in `docs/sample-data/schema.sql`, and each owner adds at least five sample rows for their tables.

Table	Key fields	Owner
staff	staff_id (PK), full_name, username, password, role, contact_no	Paramie
room	room_id (PK), room_no, type, rate, capacity, status	De Silva
guest	guest_id (PK), name, nic_passport, contact, email	Dissanayake
reservation	reservation_id (PK), guest_id (FK), room_id (FK), date_from, date_to, advance_amount, source, status	Dayarathne
booking	booking_id (PK), reservation_id (FK, nullable), guest_id (FK), room_id (FK), date_from, date_to, status	Prabath
checkin_checkout	record_id (PK), booking_id (FK), checkin_time, checkout_time, checkout_type, extra_charge	Dissanayake
invoice	invoice_id (PK), booking_id (FK), room_total, extra_charges, advance_deducted, balance_due, status	Wickramarathna
payment	payment_id (PK), invoice_id (FK, nullable), reservation_id (FK, nullable), amount, method, paid_at	Wickramarathna

Data access pattern

- `DBConnection` in the `common` package opens a JDBC connection using `application.properties`.
- Each owner writes `<Entity>Dao implements CrudDao<Entity>` using `PreparedStatement` for insert, find, update and delete, with a private `mapRow(ResultSet)` method.
- All SQL uses `?` parameters and try-with-resources, so connections always close.
- Controllers call services and services call DAOs. A controller never runs SQL directly.

05 System Architecture

Each member works inside their own Java package and JSP folder, so the GitHub history shows clearly who wrote each part. Shared files change only after the group agrees.

Shared base types

Type	Kind	Purpose
`BaseEntity`	Abstract class	Private `id` with getter/setter and abstract `getDisplayName()`. Every entity extends it.
`CrudDao<T>`	Interface	`create(T)`, `findById(int)`, `findAll()`, `update(T)`, `delete(int)`. Every DAO implements it.
`RoomAllocation`	Abstract class	Shared fields `guestId`, `roomId`, `dateFrom`, `dateTo`; methods `getNights()`, `overlaps(from, to)` and abstract `getAllocationType()`. Reservation and Booking extend it.

Integration methods

Method names are fixed in week 1. Until a component is ready, its users work against a stub with the same method names.

Provided by	Methods	Used by

Room	`RoomService.findById(id)`
	`isAvailable(roomId, from, to)`
	`setStatus(roomId, status)`
		Reservation, Booking, Check-in/out

Guest	`GuestService.findById(id)` `findOrCreate(nicPassport, ...)`	Reservation, Booking

Reservation	`ReservationService.findById(id)`
	`markConverted(id)`
	`hasOverlap(roomId, from, to)`
		Booking, Room

Booking	`BookingService.findById(id)`
	`setStatus(id, status)`
	`hasOverlap(roomId, from, to)`
		Check-in/out, Finance, Room

Check-in/out	`CheckInOutService.getExtraCharge(bookingId)`	Finance
Finance	`PaymentService.recordAdvance(reservationId, amount, method)` `getAdvancePaid(reservationId)`	Reservation
Admin	Session attribute `loggedStaff` `LoginInterceptor` protects all staff pages	All components

Project folder structure

```
hotel-reservation-system/
├── pom.xml
├── README.md
├── .gitignore
├── src/main/java/com/hotel/reservation/
│   ├── HotelReservationSystemApplication.java
│   ├── common/                                (Prabath)
│   │   ├── BaseEntity.java
│   │   ├── CrudDao.java
│   │   ├── RoomAllocation.java
│   │   ├── DBConnection.java
│   │   ├── LoginInterceptor.java
│   │   └── WebConfig.java
│   ├── admin/                                 (Paramie)
│   │   ├── Staff.java
│   │   ├── AdminUser.java
│   │   ├── Receptionist.java
│   │   ├── StaffDao.java
│   │   ├── StaffService.java
│   │   └── StaffController.java
│   ├── room/                                  (De Silva)
│   │   ├── Room.java
│   │   ├── SingleRoom.java
│   │   ├── DoubleRoom.java
│   │   ├── SuiteRoom.java
│   │   ├── RoomDao.java
│   │   ├── RoomService.java
│   │   └── RoomController.java
│   ├── reservation/                           (Dayarathne)
│   │   ├── Reservation.java
│   │   ├── PhoneReservation.java
│   │   ├── EmailReservation.java
│   │   ├── ReservationDao.java
│   │   ├── ReservationService.java
│   │   └── ReservationController.java
│   ├── booking/                               (Prabath)
│   │   ├── Booking.java
│   │   ├── ReservationBooking.java
│   │   ├── WalkInBooking.java
│   │   ├── BookingDao.java
│   │   ├── BookingService.java
```

```
│   │   └── BookingController.java
│   ├── guest/                              (Dissanayake)
│   │   ├── Guest.java
│   │   ├── GuestDao.java
│   │   ├── GuestService.java
│   │   └── GuestController.java
│   ├── checkinout/                         (Dissanayake)
│   │   ├── CheckInRecord.java
│   │   ├── CheckoutPolicy.java
│   │   ├── StandardCheckout.java
│   │   ├── LateCheckout.java
│   │   ├── EarlyCheckout.java
│   │   ├── CheckInRecordDao.java
│   │   ├── CheckInOutService.java
│   │   └── CheckInOutController.java
│   └── finance/                            (Wickramarathna)
│       ├── Invoice.java
│       ├── Payment.java
│       ├── CardPayment.java
│       ├── CashPayment.java
│       ├── InvoiceDao.java
│       ├── PaymentDao.java
│       ├── InvoiceService.java
│       ├── PaymentService.java
│       └── FinanceController.java
├── src/main/resources/
│   ├── application.properties
│   └── static/  (css, js)
├── src/main/webapp/WEB-INF/jsp/
│   ├── common/        header, footer, navbar, home
│   ├── admin/         login, dashboard, staff-register, staff-list, staff-edit
│   ├── room/          room-add, room-list, room-edit, room-search
│   ├── reservation/   reservation-new, reservation-list, reservation-edit
│   ├── booking/       booking-new, booking-list, booking-edit
│   ├── guest/         guest-form, guest-list
│   ├── checkinout/    checkin, checkout, occupancy-list
│   └── finance/       invoice-generate, invoice-view, payment-record, finance-report
└── docs/
    ├── class-diagrams/
    ├── final-report/
    └── sample-data/   schema.sql, <module>-data.sql
```

All JSP pages use the `.jsp` extension (for example `room-list.jsp`).

06 Component Plans

Every member builds in the same order and commits at each step: entity and subclasses, DAO, service, controller (one CRUD operation at a time), JSP pages, styling and testing.

1 Admin Management

Paramie W. D. | IT26100491 | `feature/IT26100491-admin-management`

Classes: abstract `Staff` with private fields, hand-written getters/setters and abstract `getPermissions()`. `AdminUser` and `Receptionist` extend it; an AdminUser can manage staff and void invoices, a Receptionist cannot. The navigation bar shows menu items from `getPermissions()`.

Operation	Endpoint	Description
Create	`POST /admin/register`	Add a staff account (admin only)
Read	`GET /admin/list` `GET /admin/search?query=`	List and search staff
Update	`POST /admin/update/{staffId}`	Edit details, role and password
Delete	`POST /admin/delete/{staffId}`	Remove a staff account (not your own)
Login	`GET/POST /admin/login` `GET /admin/logout`	Start and end the session

Pages: `login.jsp`, `dashboard.jsp`, `staff-register.jsp`, `staff-list.jsp`, `staff-edit.jsp`

2 Room Management

De Silva M.R | IT26100159 | `feature/IT26100159-room-management`

Classes: abstract `Room` with abstract `calculateRate(int nights)` and `getMaxOccupancy()`. `SingleRoom`, `DoubleRoom` and `SuiteRoom` override both (for example, a suite adds a service charge).

Operation	Endpoint	Description
Create	`GET/POST /room/add`	Add a room
Read	`GET /room/list` `GET /room/search?type=&from=&to=`	List rooms; find rooms free for given dates
Update	`GET/POST /room/update/{roomId}`	Edit rate, type, capacity or maintenance status
Delete	`POST /room/delete/{roomId}`	Remove a room (blocked if it has future reservations or bookings)

Pages: `room-add.jsp`, `room-list.jsp`, `room-edit.jsp`, `room-search.jsp`

3 Reservation Management

Dayarathne D. H. P. | IT25100116 | `feature/IT25100116-reservation-management`

Classes: abstract `Reservation` extends `RoomAllocation` with abstract `calculateRequiredAdvance(double roomTotal)`. `PhoneReservation` and `EmailReservation` override it with different advance rules.

Operation	Endpoint	Description
Create	`GET/POST /reservation/new`	Check availability, save the reservation and record the advance through Finance
Read	`GET /reservation/list?status=&query=`	List and search reservations
Update	`GET/POST /reservation/update/{id}`	Change dates or room (availability re-checked)
Delete	`POST /reservation/cancel/{id}` `POST /reservation/delete/{id}`	Cancel a no-show; delete a cancelled record
Convert	`POST /reservation/convert/{id}`	Open the Booking form with this reservation filled in

Pages: `reservation-new.jsp`, `reservation-list.jsp`, `reservation-edit.jsp`

4 Booking Management

Prabath G.O.G.D. | IT26100036 | `feature/IT26100036-booking-management`

Classes: abstract `Booking` extends `RoomAllocation` with abstract `calculateAmountDue(double roomTotal)`. `ReservationBooking` deducts the advance already paid; `WalkInBooking` charges the full amount.

Operation	Endpoint	Description
Create	`GET /booking/new?reservationId=` `POST /booking/confirm`	Confirm a converted reservation or a walk-in and assign a room
Read	`GET /booking/list?status=&query=`	List and search bookings
Update	`GET/POST /booking/update/{id}`	Change room or extend the stay (availability re-checked)
Delete	`POST /booking/cancel/{id}`	Cancel a booking before check-in

Pages: `booking-new.jsp`, `booking-list.jsp`, `booking-edit.jsp`

5 Check-in/Check-out and Guest Management

Dissanayake D.M.S.S | IT26100027 | `feature/IT26100027-checkinout-management`

Classes: `Guest` extends `BaseEntity`. `CheckInRecord` stores check-in and check-out times. Abstract `CheckoutPolicy` declares `calculateExtraCharge(CheckInRecord r, double nightlyRate)`; `StandardCheckout`, `LateCheckout` and `EarlyCheckout` override it. The policy is chosen at check-out from the actual time.

Operation	Endpoint	Description
Create	`GET/POST /guest/new` `POST /checkin/{bookingId}`	Register a guest; check in (room becomes OCCUPIED)
Read	`GET /guest/list?query=` `GET /occupancy/list`	Search guests; view current occupancy
Update	`GET/POST /guest/update/{id}` `POST /checkout/{bookingId}`	Edit a guest; check out with any extra charge (room becomes AVAILABLE)
Delete	`POST /guest/delete/{id}` `POST /checkin/void/{recordId}`	Delete a guest with no active stays; void an incorrect check-in

Pages: `guest-form.jsp`, `guest-list.jsp`, `checkin.jsp`, `checkout.jsp`, `occupancy-list.jsp`

6 Finance Management

Wickramarathna B.A.N.U.H | IT26100009 | `feature/IT26100009-finance-management`

Classes: `Invoice` extends `BaseEntity` with `calculateBalance()`. Abstract `Payment` declares `processPayment()` and `getReceiptText()`; `CardPayment` (validates card details, adds a card fee) and `CashPayment` (calculates change) override them.

Operation	Endpoint	Description
Create	`GET/POST /finance/invoice/{bookingId}` `POST /finance/payment`	Generate the final invoice; record an advance or final payment
Read	`GET /finance/payments` `GET /finance/invoice/view/{id}`	Payment history; view and print an invoice
Update	`POST /finance/invoice/update/{id}`	Add extra charges or a discount; mark PAID when the balance is zero
Delete	`POST /finance/invoice/void/{id}`	Void an incorrect invoice (admin only)

Pages: `invoice-generate.jsp`, `invoice-view.jsp`, `payment-record.jsp`, `finance-report.jsp`

07 OOP Concepts

Each component applies all five OOP concepts in its own classes. Encapsulation and information hiding follow the same rule everywhere: private fields, hand-written getters and setters with validation, and status changes only through methods such as `checkIn()` or `markConverted()`, never by setting another module's fields directly.

Member	Inheritance	Polymorphism	Abstraction
Paramie	`AdminUser`, `Receptionist` extend `Staff`	`getPermissions()`	abstract `Staff`; `StaffDao` implements `CrudDao<Staff>`
De Silva	`SingleRoom`, `DoubleRoom`, `SuiteRoom` extend `Room`	`calculateRate(nights)` `getMaxOccupancy()`	abstract `Room`; `RoomDao` implements `CrudDao<Room>`
Dayarathne	`PhoneReservation`, `EmailReservation` extend `Reservation`	`calculateRequiredAdvance()`	abstract `Reservation` extends `RoomAllocation`
Prabath	`ReservationBooking`, `WalkInBooking` extend `Booking`	`calculateAmountDue()`	abstract `Booking`, `RoomAllocation`, `CrudDao<T>`
Dissanayake	`StandardCheckout`, `LateCheckout`, `EarlyCheckout` extend `CheckoutPolicy`	`calculateExtraCharge()`	abstract `CheckoutPolicy`; `GuestDao` implements `CrudDao<Guest>`
Wickramarathna	`CardPayment`, `CashPayment` extend `Payment`	`processPayment()` `getReceiptText()`	abstract `Payment`; DAOs implement `CrudDao`

08 Development Workflow

Each member works on their own branch and commits regularly from their own GitHub account, so individual work is visible in the history.

1. Prabath creates the repository, adds the other five members as collaborators and pushes the project skeleton, `common` package, `schema.sql` and shared JSP layout to `main`.
2. Every member sets `git config user.name` and `user.email` to match their own GitHub account.
3. Each member works only on their own branch, inside their own package and JSP folder.
4. Commit messages follow `type(module): description`, for example `feat(room): implement room update`.
5. Commit after each working step: entity, DAO, each CRUD operation and each page.
6. Open a pull request into `main` when an operation works; another member reviews and merges it. Pull `main` into your branch at least twice a week.
7. `.gitignore` excludes `target/`, `.idea/` and `*.iml`; database passwords stay in a local, uncommitted properties file.

09 Timeline

The project runs for six weeks from 12 October 2026. Weeks 2 to 4 run in parallel for all members; Room, Guest and the `common` package must be usable by the end of week 2.

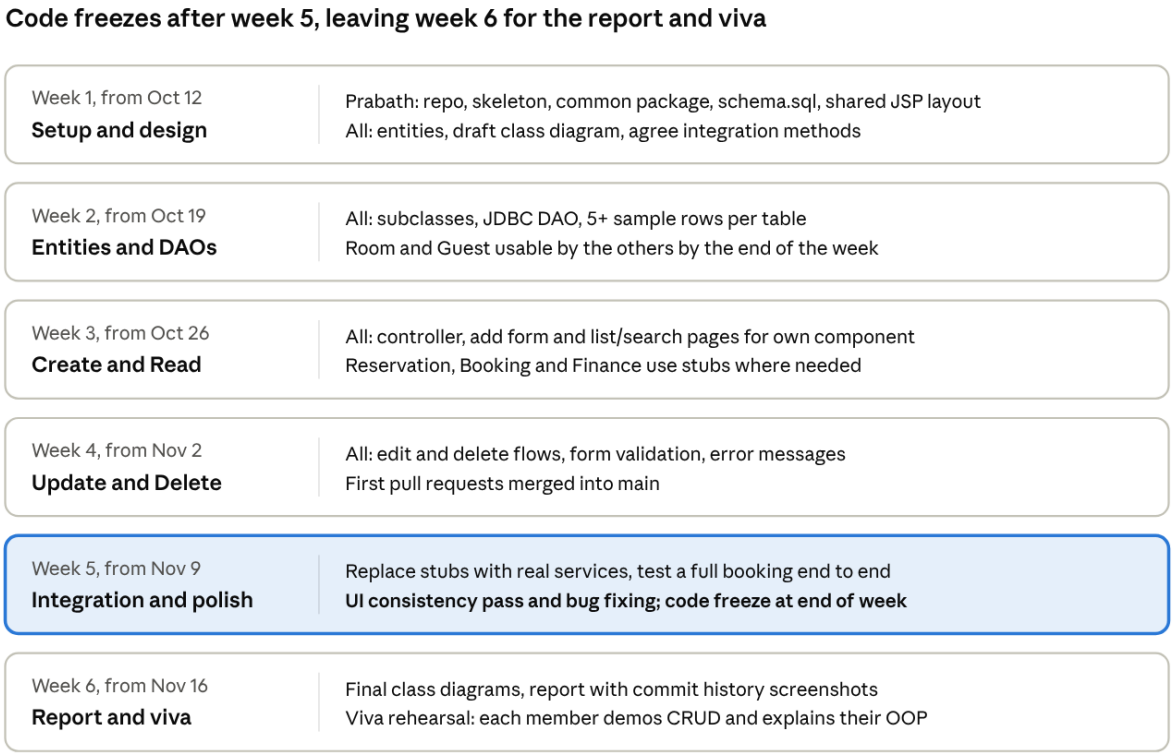


Figure 2. Six-week development timeline

10 Deliverables

Deliverable	Details	Location
Source code	All `.java`, `.jsp` and configuration files	GitHub repository
Class diagrams	One per component (abstract parent, subclasses, DAO, service, controller) plus one combined diagram showing `BaseEntity`, `RoomAllocation`, `CrudDao<T>` and the links between components	`docs/class-diagrams/`
Final report	Introduction, team table, system flow, database design, class diagrams, page screenshots, OOP explanation per member, Git commit history, conclusion	`docs/final-report/`
Sample data	Database script plus at least five realistic rows per table	`docs/sample-data/`